TypeScript

Introduce To TypeScript

Prepared by :

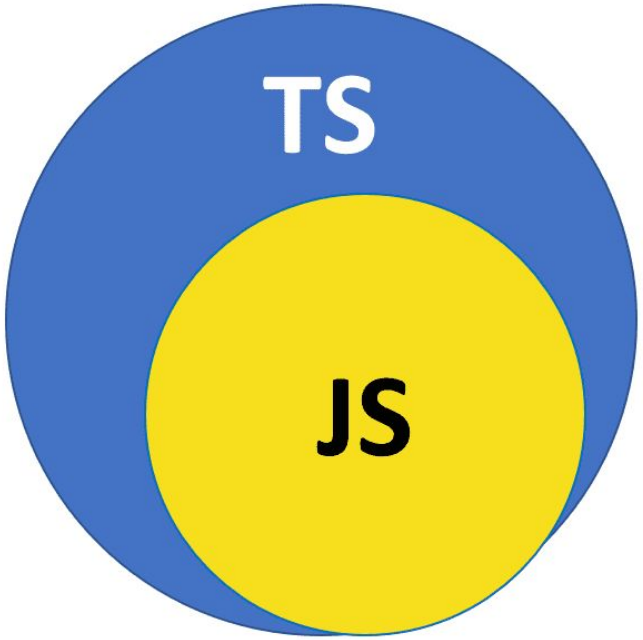
Mr. Ing Davann

Contents

1. Introduction
2. Setting up environment
3. TypeScript Basic

1. Introduction

TypeScript is a superset of JavaScript, meaning any valid JavaScript code is also valid TypeScript code. TypeScript adds optional static typing and other features on top of JavaScript, allowing developers to catch errors early in the development process and write more maintainable code.



1. Introduction (cont.)

Why is TypeScript ?

- Fewer runtime errors (catch issues early).
- Better code maintainability (easier debugging).
- Great for large-scale applications (better structure).
- Works with existing JavaScript code (can gradually adopt it).

How TypeScript Work ?

As you known **TypeScript** is a superset of JavaScript that adds static typing and other features. It compiles (or transpiles) into plain JavaScript, which browsers and Node.js can execute. TypeScript does not run directly in the browser.

1. Introduction (cont.)

Why is TypeScript ?

- Fewer runtime errors (catch issues early).
- Better code maintainability (easier debugging).
- Great for large-scale applications (better structure).
- Works with existing JavaScript code (can gradually adopt it).

How TypeScript Work ?

As you known **TypeScript** is a superset of JavaScript that adds static typing and other features. It compiles (or transpiles) into plain JavaScript, which browsers and Node.js can execute. TypeScript does not run directly in the browser.

1. Introduction (cont.)

Advantage and disadvantage of TypeScript

✅ Advantage

- **Static Typing (Fewer Bugs)** : TypeScript catches type errors at compile time, reducing runtime crashes.
- **Improved Code Readability & Maintainability** : Enforces consistent data types, making large codebases easier to maintain.
- **Better IntelliSense & Autocompletion** : TypeScript provides richer code suggestions and faster development in VS Code.
- **OOP Features (Interfaces, Classes, Generics)** : Supports interfaces, abstract classes, and generics for better structuring.

1. Introduction (cont.)

Advantage and disadvantage of TypeScript

✅ Advantage

- **Modern JavaScript Support** : TypeScript supports ES6+ features, allowing you to use the latest JavaScript features while ensuring backward compatibility.

❌ Disadvantage

- **Compilation Step Required**
  - TypeScript must be compiled into JavaScript, adding a build step.
  - You can’t run .ts files directly in the browser.

1. Introduction (cont.)

Advantage and disadvantage of TypeScript

❌ Disadvantage

- **Learning Curve** : If you're used to JavaScript, TypeScript’s strict typing, interfaces, and generics may take time to learn.
- **Not Natively Supported by Browsers** : Browsers only understand JavaScript, so TypeScript must be transpiled first.
- **Longer Development Time** : Writing types and fixing type-related errors may slow down development in small projects.
- **Increased Bundle Size** : TypeScript may generate larger JavaScript files, impacting performance if not optimized properly.

1. Introduction (cont.)

History of TypeScript

- **2010** : Microsoft identified JavaScript's scalability issues.
- **2012** (TypeScript 0.8) : First release, adding static typing, classes, and modules.
- **2015** (TypeScript 1.5) : ES6 features & Angular 2 adopted TypeScript, boosting popularity.
- **2016** (TypeScript 2.0) : Introduced strict null checks, readonly properties.
- **2019** (TypeScript 3.5) : Performance improvements, adopted by Google, Airbnb, etc.

1. Introduction (cont.)

History of TypeScript

- **2020-2023** (TypeScript 4.x) : Advanced type inference, better tooling, and editor support.
- **2024+** : TypeScript dominates modern frameworks (Next.js, Angular, NestJS) and is widely used in large-scale applications.

1. Introduction (cont.)

Ecosystems of TypeScript

The TypeScript ecosystem includes **tools**, **frameworks**, **libraries**, and a **community** of developers. These resources help developers write cleaner, more maintainable code.

🛠️ Tools

- TypeScript compiler (tsc): The official tool for transpiling TypeScript code into JavaScript
- Integrated Development Environments (IDEs): IDEs like VS Code offer features like code completion, debugging, and intellisense

1. Introduction (cont.)

Ecosystems of TypeScript

🛠️ Tools

- Build tools: Tools like Webpack and Gulp automate the compilation process
- WebStorm: A tool that helps developers work with JavaScript and TypeScript

📑 Frameworks and libraries

- The TypeScript ecosystem has a rich set of frameworks and libraries

👯 Community

- The TypeScript community is a place to ask questions, share knowledge, and learn from other developers

1. Introduction (cont.)

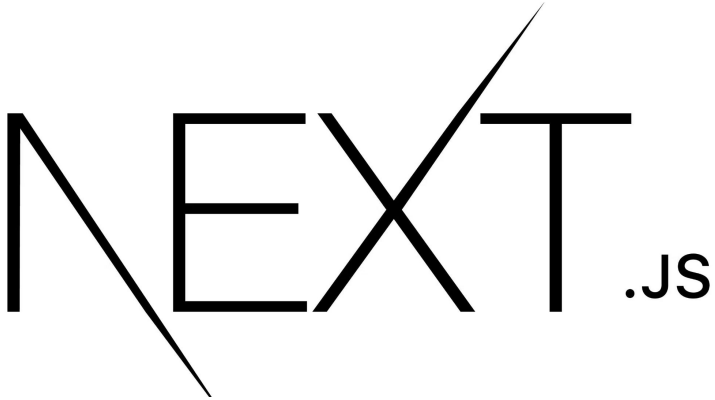
TypeScript with framework and library

TypeScript support with React, Angular, Vue.js, Express.js, etc.

React

Angular

Vue



Remix

NuxtJS

2. Setting up environment

To install TypeScript, you can use npm (Node Package Manager), which comes bundled with Node.js.

1. Install Node.js and npm :

- If you haven't already installed Node.js, download and install it from the [official Node.js https://nodejs.org/en.
- npm comes bundled with Node.js, so once you install Node.js, npm will also be installed automatically.

2. Setting up environment (cont.)

2. Install TypeScript globally :

- Once Node.js and npm are installed, open your command-line interface (CLI) or terminal. To install TypeScript globally, run the following command :

```
npm install -g typescript
```

- This command will install the latest version of TypeScript globally on your system. The -g flag installs the package globally, making it available as a command-line tool.

2. Setting up environment (cont.)

3. Verify TypeScript Installation :

- After the installation completes, you can verify that TypeScript is installed by running the following command :

```
tsc --version (for windows) | tsc -v (for linux or mac)
```

- This command will display the installed version of TypeScript. If you see the version number, TypeScript is installed successfully.

2. Setting up environment (cont.)

4. Install TypeScript Locally (Optional) :

- If you prefer to install TypeScript locally within a specific project, you can navigate to the project directory in your CLI and run the following command :

```
npm install typescript --save-dev
```

- This command will install TypeScript as a development dependency (-save-dev) within your project, and it will be available for use in that project.

2. Setting up environment (cont.)

📝 Create TypeScript config file

Make sure you're in the root folder of your project and type in your terminal

```
tsc --init
```

The file tsconfig.json will be create in the root folder of your project and in the terminal you should have an answer like that: (next slide)

2. Setting up environment (cont.)

📝 Create TypeScript config file

```
Created a new tsconfig.json with:

TS
 target: es2016
 module: commonjs
 strict: true
 esModuleInterop: true
 skipLibCheck: true
 forceConsistentCasingInFileNames: true

You can learn more at https://aka.ms/tsconfig
```

2. Setting up environment (cont.)

🛠️ Configuring the TypeScript Compiler

Here are some important configurations you should include in your tsconfig.json file for a proper TypeScript setup:

```
{
    "compilerOptions": {
      "target": "ES6",
      "module": "CommonJS",
      "outDir": "./dist",
      "rootDir": "./src",
      "strict": true,
      "esModuleInterop": true,
      "skipLibCheck": true,
      "forceConsistentCasingInFileNames": true
    },
    "include": ["src"],
    "exclude": ["node_modules"]
}
```

2. Setting up environment (cont.)

🛠️ Configuring the TypeScript Compiler

Important Compiler Options

- `target: "ES6"` -> Defines the JavaScript version to compile to (e.g., "ES5", "ES6", "ESNext").
- `module: "CommonJS"`

  -> Defines the module system for importing/exporting files.

  -> Use "ESNext" for ESM modules.

- `outDir: "./dist"` -> Specifies where the compiled JavaScript files will be stored.

2. Setting up environment (cont.)

🛠️ Configuring the TypeScript Compiler

Important Compiler Options (cont.)

- `rootDir: "./src"` -> Specifies where your TypeScript source files are located.
- `strict: true` -> Enables all strict type-checking options.
- `esModuleInterop: true` -> Allows interoperability with CommonJS and ES modules.
- `skipLibCheck: true` -> Skips type checking of declaration files (.d.ts), improving compilation speed.
- `forceConsistentCasingInFileNames: true` -> Ensures that imports use consistent casing to prevent issues in case-sensitive file systems.

2. Setting up environment (cont.)

🛠️ Configuring the TypeScript Compiler

Important Compiler Options (cont.)

- `include: ["src"]` -> Specifies which files/folders to include in the compilation.
- `exclude: ["node_modules"]` -> Specifies which files/folders to exclude.

✅ How to Compile TypeScript

- Run the TypeScript compiler : `tsc` | This will generate JavaScript files inside the `dist/ folder`.
- Run the compiled file: `node dist/index.js`

2. Setting up environment (cont.)

📁 Project Structure

A well-structured TypeScript project makes development easier to manage, scalable, and maintainable.

```
├── dist/
│   ├── index.js <- Compiled JavaScript file
├── src/
│   ├── index.ts <- TypeScript source file
│   ├── tsconfig.json <- TypeScript configuration file
```

2. Setting up environment (cont.)

📁 Project Structure

🔹 Explanation

`src/index.ts`

- This is the TypeScript source file where you write your TypeScript code.
- It gets compiled into JavaScript.

`tsconfig.json`

- This file configures the TypeScript compiler (`tsc`).
- It specifies how TypeScript should be compiled into JavaScript.

2. Setting up environment (cont.)

📁 Project Structure

🔹 Explanation (cont.)

`dist/index.js`

- This is the compiled JavaScript output from `index.ts`.
- The `dist/` folder is typically used to store compiled files.

🔹 How Compilation Works

you can run by type `tsc` in your terminal. So TypeScript reads `tsconfig.json`, compiles `index.ts`, and generates `index.js` inside the `dist/` folder.

3. TypeScript Basics

1️⃣ TypeScript Syntax

TypeScript extends JavaScript by adding type annotations.

```
function hello(name: string): string {
  return `Hello, ${name}!`;
}

console.log(hello("TypeScript"));
```

3. TypeScript Basics (cont.)

2️⃣ Variables in TypeScript

TypeScript supports let, const, and var (same as JavaScript).

But it also allows type annotations:

```
let message: string = "Hello, TypeScript!";
const age: number = 25;
var isStudent: boolean = true;
```

- let: Block-scoped (preferred).
- const: Immutable (cannot be reassigned).
- var: Function-scoped (not recommended).

3. TypeScript Basics (cont.)

2️⃣ Variables in TypeScript

Type Inference : If you don't specify a type, TypeScript infers it automatically:

```
let city = "New York";  // TypeScript infers `city` as a string
city = 100;  // ❌ Error: Type 'number' is not assignable to type 'string'
```

3️⃣ **Data Types** : TypeScript provides several built-in types

- **Primitive Types** :

Primitive include type `string`, `number`, `boolean`, `null` and `undefined`.

3. TypeScript Basics (cont.)

3️⃣ Data Types

- **Special Types** :

1. **Any** (any can hold any type, but avoid using it)

```
let data: any = "Hello";
data = 42;  // ✅ Allowed
data = true;  // ✅ Allowed
```

3. TypeScript Basics (cont.)

3️⃣ Data Types

- **Special Types** :

2. **Union** Type (|)

```
let id: string | number;
id = "123";  // ✅ OK
id = 123;  // ✅ OK
id = true;  // ❌ Error
```

3. TypeScript Basics (cont.)

3 Data Types

- **Special Types** :

3. **Array**

```
let numbers: number[] = [1, 2, 3, 4];
let names: string[] = ["Alice", "Bob"];
```

4. **Tuples** : Fixed-length arrays with different types

```
let person: [string, number] = ["John", 25];
```

3. TypeScript Basics (cont.)

3 Data Types

- **Special Types** :

5. **Enums** : Useful for defining named constants:

```
enum Role { Admin, User, Guest }
let myRole: Role = Role.Admin;
console.log(myRole); // Output: 0 (index of Admin)
```

3. TypeScript Basics (cont.)

3 Data Types

- **Special Types** :

6. Objects

```
let user: { name: string; age: number } = {
    name: "Alice",
    age: 30
};
```

3. TypeScript Basics (cont.)

3 Data Types

- **Special Types** :

7. Functions with Types

```
function add(a: number, b: number): number {
    return a + b;
}
console.log(add(5, 10)); // Output: 15
```

3. TypeScript Basics (cont.)

3 Data Types

- **Special Types** :

8. Type Aliases

```
type User = { name: string; age: number };
let user1: User = { name: "Bob", age: 28 };
```

3. TypeScript Basics (cont.)

4 Control Flow Structures

TypeScript supports standard control structures like JavaScript :

If-Else Statement

```
let num: number = 10;
if (num > 0) {
  console.log("Positive number");
} else if (num < 0) {
  console.log("Negative number");
} else {
  console.log("Zero");
}
```

3. TypeScript Basics (cont.)

4 Control Flow Structures

Switch Statement

```
let day : number = 3;
switch(day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    default:
        console.log("Other day");
}
```

3. TypeScript Basics (cont.)

4 Control Flow Structures

Loops

```
// For loop
for (let i = 0; i < 5; i++) {
    console.log(i);
}
// While Loop
let count: number = 0;
while (count < 3) {
  console.log(count);
  count++;
}
```

3. TypeScript Basics (cont.)

4 Control Flow Structures

Loops (cont.)

```
// Do While loop
let num2: number = 5;
do {
  console.log(num2);
  num2--;
} while (num2 > 0);
// For-Of Loop (for array)
let colors: string[] = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color);
}
```

3. TypeScript Basics (cont.)

4 Control Flow Structures

Loops (cont.)

```
let person: { name: string; age: number } = { name: "Alice", age: 25 };

for (let key in person) {
  console.log(`${key}: ${person[key as keyof typeof person]}`);
}
```

Resources

- https://www.typescriptlang.org/
- https://thiraphat-ps-dev.medium.com/best-practices-for-structuring-a-react-typescript-project-f5ee7f9a264e
- https://medium.com/@olofbaage/typescript-in-a-nutshell-how-to-set-your-first-project-e979eaf5454b
-

សូមអរគុណ!

We come for Cambodia, for our education